Business Requirements Document (BRD)

Project: Customer Orders ETL and Reporting (Delta Lake / Delta Live Tables)

Prepared by: Senior Business Analyst

Date: 2026-03-30

Version: 1.0

1. Executive Summary

- Purpose: Build a robust, auditable, and production-ready ETL pipeline that ingests customer and order CSV data, cleans and enriches it, produces an SCD Type 2 order summary, and derives customer-level aggregate spend. The implementation will use Databricks Delta Lake with Delta Live Tables (DLT).
- Goals:
- Load source CSVs into Delta tables (customer, order).
- Apply data quality: remove nulls and duplicates.
- Enrich order data by computing TotalAmount.
- Create and maintain an ordersummary table with SCD Type 2 behavior for customer changes.
- Create customeraggregatespend summary aggregated by customer name and date.
- Use Delta Live Tables for implementation to support orchestration, lineage, monitoring, and incremental processing.

2. Scope

- In scope:
- Ingestion of CSVs from specified volumes (paths) into Delta tables: customer and order.
- Schema enforcement for source tables based on provided fields.
- Data cleansing: removal of nulls and duplicates in both tables.
- Computation of TotalAmount on orders: PricePerUnit * Qty.
- Creation (if not exists) of ordersummary (SCD Type 2) and customeraggregatespend tables in catalog gen_ai_poc_databrickscoe, schema sdlc_wizard.
- SCD Type 2 behavior for customer changes: maintain StartDate, EndDate, and ActiveFlag, mark historical rows inactive, insert new active rows.
- Aggregate TotalAmount by Name and Date from ordersummary into customeraggregatespend (Name, TotalAmount, Date).
- Implementation using Delta Live Tables with incremental and governance features.
- Out of scope:
- Changes to source systems that produce CSVs.
- Downstream dashboarding or BI development (unless requested later).
- Schema evolution beyond fields listed unless agreed in change control.

- Complex enrichment beyond TotalAmount and SCD attributes.

3. Business Requirements (High-level)

BR-1. Source Ingestion

- The solution must read CSV files from:
- /Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/customerdata → Delta table: customer
- /Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/orderdata → Delta table: order
- Ingestion must be idempotent and support incremental updates (new/modified files).

BR-2. Source Schemas (enforced)

- customer table fields:
- CustId (primary business key)
- Name
- EmailId
- Region
- order table fields:
- OrderId (unique identifier per order)
- ItemName
- PricePerUnit (numeric/decimal)
- Qty (numeric/integer)
- Date (date/datetime)
- CustId (foreign key to customer.CustId)

BR-3. Computed Field

- Add TotalAmount to the order table: TotalAmount = PricePerUnit * Qty.
- Ensure numeric precision and scale are defined to avoid rounding errors.

BR-4. Data Quality

- Remove null records and duplicate records from both customer and order tables.
- Null removal: define required fields per table (see section 5).
- Duplicate removal: deduplicate by primary key (customer: CustId; order: OrderId). For orders, deduplication may prefer latest file timestamp or latest record; define tie-breaker rule.

BR-5. ordersummary Table

- Create table ordersummary in catalog gen_ai_poc_databrickscoe, schema sdlc_wizard if it does not exist.
- ordersummary logical schema (base fields):

- CustId
- Name
- EmailId
- Region
- OrderId
- ItemName
- PricePerUnit
- Qty
- Date
- Additionally, ordersummary must include SCD Type 2 operational fields:
- StartDate (timestamp) — when this version became effective
- EndDate (timestamp, nullable) — when this version was superseded; null for active rows
- ActiveFlag (boolean) — true/false indicating active version
- LoadTimestamp (timestamp) — ingestion/processing time
- HashKey or RecordChecksum (optional) — for change detection on customer attributes

BR-6. SCD Type 2 Behavior

- When a customer record changes (Name, EmailId, Region) for the same CustId, the system shall:
- Identify existing active records for that CustId in ordersummary.
- Set ActiveFlag = false and EndDate = processing timestamp (or previous Date/time granularity) on existing active records.
- Insert new records representing the new customer details joined with order data with StartDate = processing timestamp, EndDate = null, ActiveFlag = true.
- Changes limited to customer attributes shall trigger SCD Type 2 behavior in ordersummary; changes to order transactional data (OrderId, ItemName, PricePerUnit, Qty, Date) should be handled as per business rules (see assumptions / rules).
- Maintain full history of customer attribute changes.

BR-7. customeraggregatespend Table

- Create table customeraggregatespend in catalog gen_ai_poc_databrickscoe, schema sdlc_wizard if it does not exist.
- Schema:
- Name
- TotalAmount (sum of TotalAmount)
- Date
- Aggregation rules:
- Source: ordersummary (only active or all versions? See business rule below)

- Group by Name and Date
- Aggregate TotalAmount using sum.
- Business decision required: whether aggregation must use active current customer Name as of order Date, or use the name in effect when the order occurred. Recommendation: aggregate based on ordersummary row effective at order Date (SCD-aware).

BR-8. Implementation Platform

- Use Delta Live Tables (DLT) to implement steps 1–10 for:
- Declarative pipeline definitions
- Automated orchestration and retries
- Data quality policies and expectations
- Lineage, monitoring, and incremental processing
- Provide DLT pipeline configurations to read from source volumes, apply transformations, manage SCD logic, and write target Delta tables.

4. Functional Requirements (detailed)

FR-1. Ingest Source CSVs

- Implement DLT source tables reading CSV with explicit schema enforcement, header handling, delimiter detection, and timestamp/partition handling for incremental ingestion.
- Must support file format changes control and schema drift notifications.

FR-2. Schema Enforcement and Casting

- Cast PricePerUnit and Qty to numeric types (decimal or double for price; integer for Qty).
- Parse Date to date or timestamp type using provided format (business to confirm format, e.g., yyyy-MM-dd or ISO8601).
- Reject or quarantine records that fail type casting into a designated bad records path or quarantine table, logged with error reason.

FR-3. Compute TotalAmount

- For each order row, compute TotalAmount = PricePerUnit * Qty.
- Define precision/scale: e.g., decimal(18,2) or configurable.

FR-4. Data Cleansing

- Define required fields:
- customer: CustId required, Name or EmailId required (business to confirm which are mandatory)
- order: OrderId, CustId, PricePerUnit, Qty, Date required
- Drop or quarantine rows with nulls in required fields.
- Deduplicate:

- customer: deduplicate by CustId, keeping latest record based on file ingestion timestamp or provided update timestamp (if none, use last-writes-wins by processing order).
- order: deduplicate by OrderId, keeping latest record similarly.
- Record deduplication and null-removal counts in monitoring/metrics.

FR-5. Create / Maintain ordersummary (SCD Type 2)

- When loading ordersummary:
- Join cleansed customer and order tables on CustId.
- For each joined row, detect if customer attributes (Name, EmailId, Region) differ from the active ordersummary record for same CustId and OrderId (or if no active record exists).
- If no existing active record for the transaction, insert new active row with StartDate = processing_time, EndDate = null, ActiveFlag = true.
- If existing active record exists and customer attributes are unchanged, update other fields if needed (e.g., if order transactional data changed); business to define whether order changes should create a new SCD row. Recommended: only customer attribute changes drive SCD Type 2; order transactional changes should update transactional columns within the active SCD row or be treated as new transactional entry depending on OrderId uniqueness.
- If customer attributes changed, update existing active rows: set ActiveFlag=false and EndDate=processing_time, and insert new row with updated attributes and ActiveFlag=true.
- Ensure SCD operations are atomic and idempotent, ideally using Delta MERGE with appropriate matching condition and versioning.

FR-6. SCD Fields and Defaults

- StartDate default: processing/insertion timestamp
- EndDate default: null for active; set to processing timestamp when row is superseded
- ActiveFlag: boolean true for StartDate <= processing_time and EndDate is null
- LoadTimestamp: processing timestamp for traceability
- Optional: include RecordChecksum (hash of customer attribute fields) to simplify change detection.

FR-7. Aggregation into customeraggregatespend

- Build aggregation pipeline:
- Source: ordersummary (use the SCD-effective record appropriate to the business requirement).
- Group by Name and Date.
- Sum TotalAmount into TotalAmount column.
- Write result to customeraggregatespend (overwrite or upsert strategy to support incremental loads—recommend MERGE by Name and Date).
- Date granularity: business to confirm (day-level recommended; if time included, truncate to date).
- Backfill: support initial full backfill and subsequent incremental updates.

FR-8. Delta Live Tables Implementation Requirements

- Use DLT to define stream/batch tables and pipeline.
- Define expectations (data quality rules) for nulls, duplicates, and type constraints; fail or quarantine per policy.
- Use DLT-managed tables for customer, order, ordersummary, and customeraggregatespend, under the specified catalog/schema.
- Configure DLT to provide metrics, alerting, and automated retries.
- Provide modular DLT notebooks or declarative pipeline code with unit tests for each transformation.

5. Non-functional Requirements

NFR-1. Performance & Scalability

- Pipeline should scale to handle growth in daily file volume (initial sizing to be determined).
- Delta tables must support efficient MERGE operations and partitioning strategy (e.g., partition ordersummary and customeraggregatespend by Date or Year/Month/Date).

NFR-2. Data Quality & Observability

- Implement monitoring dashboards for:
- Row counts ingested per source
- Null/quarantine counts
- Duplicate counts removed
- SCD changes detected and applied
- Aggregation metrics
- Retain lineage for troubleshooting.

NFR-3. Security & Access Control

- Store target tables in catalog gen_ai_poc_databrickscoe, schema sdlc_wizard.
- Apply least privilege access; grant write to data engineering service account; read to analytics roles as required.
- Encrypt data at rest and in transit per organizational policies.

NFR-4. Reliability & Recovery

- Ensure idempotent processing and transactional writes using Delta Lake ACID guarantees.
- Implement a retry strategy for transient failures; keep error logs and quarantine datasets for bad records.
- Maintain retention policy for historical data and archived files.

NFR-5. Compliance & Retention

- Retain SCD history per business/legal retention requirements (TBD).

- Support data deletion/rectification requests by tracking source lineage and load timestamps.

6. Data Model and Schemas (Final Target)

6.1 Source tables (Delta)

- customer (Delta)
- CustId: string (primary business key)
- Name: string
- EmailId: string
- Region: string
- IngestionTimestamp: timestamp (DLT-managed)
- order (Delta)
- OrderId: string
- ItemName: string
- PricePerUnit: decimal(18,2)
- Qty: integer
- Date: date (or timestamp)
- CustId: string (foreign key)
- TotalAmount: decimal(18,2) — computed PricePerUnit * Qty
- IngestionTimestamp: timestamp

6.2 ordersummary (Delta, SCD Type 2)

- CustId: string
- Name: string
- EmailId: string
- Region: string
- OrderId: string
- ItemName: string
- PricePerUnit: decimal(18,2)
- Qty: integer
- Date: date
- TotalAmount: decimal(18,2)
- StartDate: timestamp
- EndDate: timestamp (nullable)
- ActiveFlag: boolean
- LoadTimestamp: timestamp

- RecordChecksum: string (optional for change detection)
- SourceFileId / SourceIngestionId: string (optional, for traceability)

6.3 customeraggregatespend (Delta)

- Name: string
- TotalAmount: decimal(18,2)
- Date: date
- AggregationTimestamp: timestamp (when aggregate was computed)

7. Business Rules, Assumptions, and Open Questions

Business Rules

- BRule-1: CustId is the business key for customer SCD operations.
- BRule-2: OrderId uniquely identifies an order row. Duplicate detection should keep the latest by ingestion timestamp.
- BRule-3: SCD Type 2 is driven by changes in customer attributes (Name, EmailId, Region). Order transactional changes do not create a new SCD row unless customer attributes also changed (subject to business confirmation).
- BRule-4: Aggregations will use the SCD-effective customer Name for each order at the time of the order (recommended).

Assumptions

- Source CSVs include headers and consistent field naming as provided.
- Date format is consistent; if not, pipeline will include parsing logic and quarantine errors.
- Data volumes are moderate initially but may grow.
- There is an operational Databricks workspace with Delta Live Tables enabled and permissions to create tables in gen_ai_poc_databrickscoe.sdlc_wizard.

Open Questions (require stakeholder decisions)

- Q1: Which fields are strictly required (cannot be null) for customer and order? Confirm mandatory fields to support null-removal rules.
- Q2: For deduplication, what is the tie-breaker: file ingestion timestamp, a source-provided update timestamp, or another rule?
- Q3: Should order transactional changes trigger SCD Type 2 rows in ordersummary, or should SCD be limited exclusively to customer attribute changes?
- Q4: Aggregation granularity for Date: should it be date-only, month-level, or include time? Should aggregations be computed on active records only or on historical SCD versions keyed by order date?
- Q5: Retention policy for SCD history and aggregated tables?

- Q6: Precision/scale required for PricePerUnit and TotalAmount beyond the suggested decimal(18,2)?

8. Implementation Design Overview

- Ingest stage (DLT)
- DLT tables: raw_customer_csv, raw_order_csv reading from provided volumes with schema enforcement.
- Apply parsing, type casting, and quarantine for bad records.
- Cleansing stage
- Transform raw_customer_csv → customer (deduped, nulls removed).
- Transform raw_order_csv → order (compute TotalAmount, deduped, nulls removed).
- SCD merge stage
- Join customer and order on CustId to prepare enriched records.
- Use Delta MERGE into ordersummary with SCD Type 2 logic:
- Matching keys: CustId + OrderId (or CustId alone if multiple orders map differently; confirm business key).
- When matched and customer attributes changed → set prior ActiveFlag=false, EndDate = processing_time; insert new row.
- When not matched → insert new active row.
- Aggregation stage
- Aggregate ordersummary by Name and Date → MERGE into customeraggregatespend.
- Orchestration and scheduling
- DLT pipeline scheduled (or streaming) to process new files continuously or at configured intervals.
- Monitoring
- Implement expectations and notify on failures, quarantined record counts, unusual data patterns.

9. Test Strategy

- Unit tests on sample datasets:
- Validate TotalAmount calculation.
- Validate null and duplicate removal.
- Validate SCD behavior: create change scenarios for Name/Email/Region and confirm history rows created with StartDate/EndDate/ActiveFlag.
- Validate aggregation correctness and idempotency.
- Integration tests:
- End-to-end run from CSV ingestion to final aggregate.
- Failure and retry scenarios with transient errors.
- Performance tests:

- Load tests simulating increased daily file sizes.

10. Deployment & Runbook

- Deployment steps:
- Provision DLT pipeline definitions and connectors.
- Create target catalog/schema if not present.
- Deploy notebooks/SQL code for DLT definitions.
- Run initial full backfill to create base ordersummary and customeraggregatespend.
- Runbook items:
- How to reprocess a bad file (reingest and reconcile).
- How to handle schema changes: update DLT code, run backfill if necessary.
- How to query SCD history and current active view.
- How to restore table state via Delta time travel if needed.

11. Security, Governance & Compliance

- Ensure access controls on catalog gen_ai_poc_databrickscoe.sdlc_wizard; only authorized roles may write or alter tables.
- Enable table-level and column-level masking where required for PII (EmailId may be sensitive).
- Audit logging of data modifications via Delta transaction logs.
- Data retention and deletion flows to comply with regulatory requests.

12. Acceptance Criteria

- AC-1: customer and order tables are populated from CSVs with correct schema, with nulls and duplicates removed as per rules.
- AC-2: order.TotalAmount is correctly computed for all orders.
- AC-3: ordersummary exists and implements SCD Type 2 semantics; historical customer attribute changes are preserved with correct StartDate, EndDate, and ActiveFlag.
- AC-4: customeraggregatespend contains aggregated TotalAmount by Name and Date and is consistent with ordersummary.
- AC-5: Implementation executed in Delta Live Tables, with monitoring, data quality expectations, and lineage enabled.
- AC-6: All open questions resolved and documented, or exceptions approved, before production go-live.

13. Risks & Mitigations

- Risk: Source CSV schema drift causing pipeline failures.
- Mitigation: Implement schema validation and quarantine with alerts; adopt schema evolution procedures.

- Risk: Incorrect SCD keys or merge logic causing historical corruption.
- Mitigation: Thorough testing of merge conditions; maintain backups and ability to time-travel to previous versions.
- Risk: High-volume data causing MERGE performance issues.
- Mitigation: Partitioning strategy, optimized MERGE patterns, or incremental micro-batching.
- Risk: PII exposure (EmailId)
- Mitigation: Masking/encryption and access control.

14. Deliverables

- DLT pipeline definitions (notebooks or declarative JSON/SQL).
- Delta table definitions and initial schema migrations.
- Test datasets and automated tests.
- Monitoring dashboards and alerts configuration.
- Runbook and operational documentation.
- BRD sign-off document.

15. Timeline & Estimated Effort (high-level)

- Requirements finalization & design: 1–2 weeks (including answers to open questions).
- Development (DLT pipelines, SCD logic, aggregation): 2–4 weeks.
- Testing & validation (unit, integration, performance): 1–2 weeks.
- Deployment & cutover (including initial backfill): 1 week.
- These estimates assume an available Databricks environment and data engineering resources.

16. Change Control

- Any addition to source schema, new fields for SCD triggers, or change in aggregation logic must go through formal change control with impact analysis and regression test plans.

Appendix A — Mapping Summary (from raw JSON)

- Source paths:
- customer: /Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/customerdata
- order: /Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/orderdata
- Source schemas:
- customer: [CustId, Name, EmailId, Region]
- order: [OrderId, ItemName, PricePerUnit, Qty, Date, CustId]
- Transformations:
- order.TotalAmount = PricePerUnit * Qty
- Remove nulls and duplicates in both tables

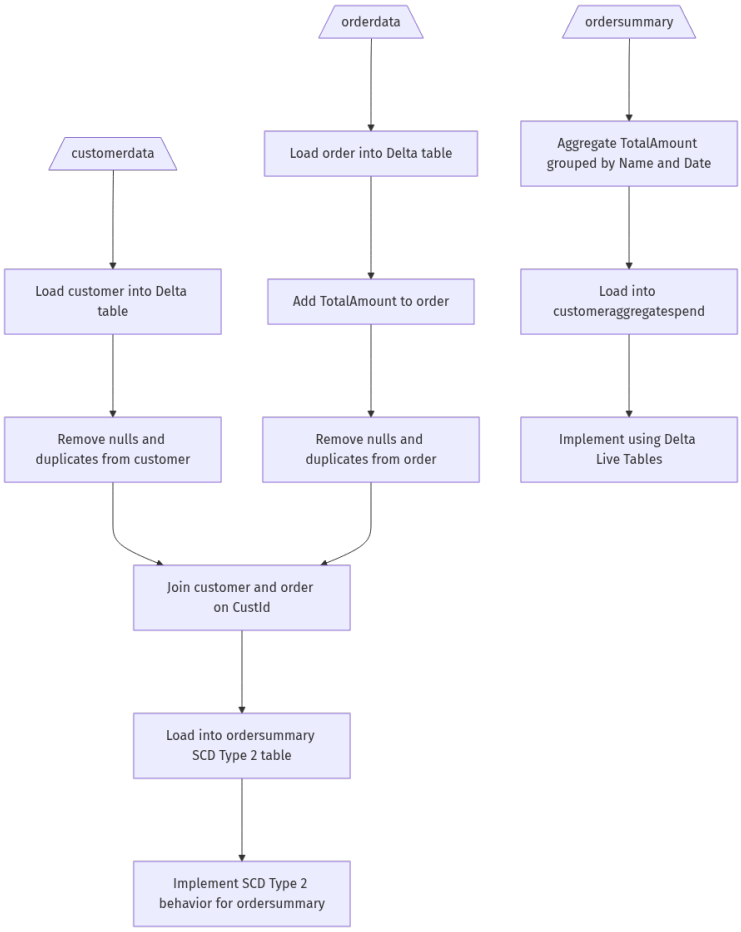
- Join customer & order on CustId → ordersummary (SCD Type 2)
- ordersummary schema: [CustId, Name, EmailId, Region, OrderId, ItemName, PricePerUnit, Qty, Date] + SCD fields
- SCD Type 2 fields: StartDate, EndDate, ActiveFlag
- customeraggregatespend schema: [Name, TotalAmount, Date] aggregated sum(TotalAmount) grouped by Name and Date
- Implementation: Delta Live Tables

Appendix B — Action Items / Next Steps

- Confirm open questions listed in section 7 with business stakeholders.
- Agree on required field lists and deduplication tie-break rules.
- Finalize numeric precision and date format.
- Provision workspace and permissions for DLT and Delta tables.
- Schedule kick-off with data engineering to begin pipeline development.

End of Document.

Data Flow Diagram (DFD)



```mermaid

graph TD;

CustomerData[/customerdata\] ---> LoadCustomer[Load customer into Delta table];

OrderData[/orderdata\] ---> LoadOrder[Load order into Delta table];

LoadCustomer ---> RemoveNullsAndDuplicatesCustomer[Remove nulls and duplicates from customer];

LoadOrder ---> AddTotalAmount[Add TotalAmount to order];

AddTotalAmount ---> RemoveNullsAndDuplicatesOrder[Remove nulls and duplicates from order];

RemoveNullsAndDuplicatesCustomer ---> JoinCustomerOrder[Join customer and order on CustId];

RemoveNullsAndDuplicatesOrder ---> JoinCustomerOrder;

JoinCustomerOrder ---> LoadOrderSummary[Load into ordersummary SCD Type 2 table];

LoadOrderSummary ---> ImplementSCDType2[Implement SCD Type 2 behavior for ordersummary];

ordersummary[/ordersummary\] ---> AggregateTotalAmount[Aggregate TotalAmount grouped by Name and Date];

AggregateTotalAmount ---> LoadCustomerAggregateSpend[Load into customeraggregatespend];

LoadCustomerAggregateSpend ---> DeltaLiveTables[Implement using Delta Live Tables];

```